

Velocity-Threshold Trigger Node

Simulated Validation and Path to Non-Simulated Deployment

1. Purpose

This node connects the timing-validation work completed earlier to a real trigger condition: instead of a manufactured condition, it watches a robot's velocity and fires a release trigger the moment that velocity crosses a set threshold. This document explains how the node was tested in a simulated environment, what the results show, and how the same setup can be moved to a non-simulated environment once the extraction arm is available.

2. How the Node Works

The `velocity_trigger_node` subscribes to a velocity topic (`nav_msgs/Odometry`, by default `/odom`) and continuously checks the incoming linear speed against a configurable threshold (`velocity_threshold`, default 0.5 m/s). The moment the threshold is crossed, two things are recorded and logged, in the same style as the earlier timing-validation node:

- Condition-met timestamp - the instant the velocity reading crosses the threshold.
- Action-sent timestamp - the instant the node publishes the release-trigger message.

The delay between these two timestamps is the same metric used for the earlier timing-validation results, so the two pieces of work can be compared directly.

3. How the Simulated Test Worked

Because the extraction arm is not yet available, the node could not be tested against a real velocity source. Instead, a test script (`test_velocity_trigger.py`) was used to simulate one. The script does not read any real sensor or physical motion; it manufactures a velocity value in software, starting at 0 m/s and increasing it in small steps, and publishes that value onto the same `/odom` topic the node listens to:

```
speed += ramp_step    # a plain number increasing each cycle
odom.twist.twist.linear.x = speed
publisher.publish(odom)
```

The script records the moment the simulated speed crosses the threshold, then listens for the node's release-trigger message and records when it arrives. This proves the node's own logic, detecting a threshold crossing and firing a trigger, works correctly and consistently, without needing any physical hardware to be present.

4. Results - Test Script Measurements

Five trials were run with a threshold of 0.5 m/s. Each trial measured the delay between the simulated velocity crossing the threshold and the node's trigger message being received back at the test script:

Trial	Delay (ms)
1	2.43
2	3.37
3	2.84
4	2.47
5	2.64

Range: 2.43ms – 3.37ms **Average:** 2.75ms

5. Results - Node-Side Log

The node also logs its own condition-met and action-sent timestamps independently of the test script, measuring only the time inside the node itself (detecting the crossing to publishing the trigger), without the extra messaging round-trip back to the test script. These five trials were read directly from the node's terminal output:

Trial	Delay (ms)
1	1.01
2	1.06
3	1.49
4	1.02
5	1.23

Range: 1.01ms – 1.49ms **Average:** 1.16ms

The node-side delay is smaller than the test-script-side delay because the node's own log only captures the time to detect and publish, while the test script's measurement also includes the messaging round-trip between the two separate processes. Both being small and consistent across all five trials confirms the delay is not hiding anywhere unexpected.

6. What This Result Does and Does Not Prove

The simulated test proves that the node's own logic is correct: it detects a threshold crossing and reliably fires a trigger with a small, repeatable delay. It does not yet prove anything about how the node behaves with a real velocity source, because the test script supplies clean, evenly-spaced, predictable numbers rather than a genuine sensor reading. A real velocity source may differ in ways the simulated test cannot capture, including:

- Sensor noise or jitter in the reported velocity, which could cause the threshold to be crossed briefly and by mistake.
- Irregular or slower message timing from the real sensor or CV-motion planner, compared to the steady rate used in the test.
- Real-world processing or network delay on the physical system that the test environment does not have.
- A threshold value (0.5 m/s) that was chosen as a placeholder and may need to be recalibrated once real velocity data is available.

7. Path to Non-Simulated Deployment

The node was deliberately built so that moving from the simulated test to a real environment requires no code changes, only a configuration change. The velocity source is set through a ROS2 parameter (`velocity_topic`), currently pointed at the test script's `/odom` topic. Once the extraction arm and its CV-motion planner are available, the same node can be pointed at the real velocity topic instead:

```
ros2 run timing_validation velocity_trigger_node \
  --ros-args -p velocity_topic:=/real_velocity_topic \
  -p velocity_threshold:=0.5
```

The recommended process for that stage is:

- Confirm the real velocity topic name and message type with `ros2 topic list` and `ros2 topic info`, and update the node's parameters to match.
- Re-run the same five-trial methodology against the real topic, recording delay results in the same format as above so the simulated and non-simulated results can be compared directly.
- Review whether the 0.5 m/s threshold is still appropriate once real velocity behaviour is visible, and adjust the `velocity_threshold` parameter if needed.
- Watch for noise-driven false triggers in the real data; if they occur, a simple debounce (requiring the threshold to be held for a short duration before firing) can be added without changing the node's overall structure.

8. Conclusion

With no extraction arm currently available, the simulated test is the appropriate way to validate the node at this stage, and the results, cross-checked from both the test script and the node's own log, show the node's core logic works correctly and consistently. The node's design keeps the real-world velocity source as a simple configuration change rather than a rebuild, so the team is positioned to gather non-simulated results as soon as the physical hardware is available.

9. Evidence

```
colcon build [1/1 done] [0 ong] X + v
"""
test_velocity_trigger.py

Publishes a rising velocity ramp on /odom and listens on /release_trigger
#!/usr/bin/env python3
"""
velocity_trigger_node.py

Standalone ROS2 node that watches a velocity topic and fires a "release
jerem@Jeremy:~/timing_ws$ cd ~/timing_ws
colcon build --packages-select timing_validation
source install/setup.bash
ros2 run timing_validation velocity_trigger_node
Starting >>> timing_validation
Finished <<< timing_validation [3.15s]

Summary: 1 package finished [3.64s]
[INFO] [1789990976.051496527] [velocity_trigger_node]: Watching '/odom' for |v| >= 0.5 m/s -> publishing on '/release_tr
igger'
[INFO] [1789991149.220909555] [velocity_trigger_node]: Condition met: speed=0.550 m/s >= threshold=0.5 m/s
[INFO] [1789991149.222127076] [velocity_trigger_node]: Action sent. Delay (condition-met -> action-sent): 1.01 ms
[INFO] [1789991149.939923825] [velocity_trigger_node]: Condition met: speed=0.550 m/s >= threshold=0.5 m/s
[INFO] [1789991149.941235076] [velocity_trigger_node]: Action sent. Delay (condition-met -> action-sent): 1.06 ms
[INFO] [1789991150.658665807] [velocity_trigger_node]: Condition met: speed=0.550 m/s >= threshold=0.5 m/s
[INFO] [1789991150.659556231] [velocity_trigger_node]: Action sent. Delay (condition-met -> action-sent): 1.49 ms
[INFO] [1789991151.378068516] [velocity_trigger_node]: Condition met: speed=0.550 m/s >= threshold=0.5 m/s
[INFO] [1789991151.379244774] [velocity_trigger_node]: Action sent. Delay (condition-met -> action-sent): 1.02 ms
[INFO] [1789991152.098105520] [velocity_trigger_node]: Condition met: speed=0.550 m/s >= threshold=0.5 m/s
[INFO] [1789991152.099348062] [velocity_trigger_node]: Action sent. Delay (condition-met -> action-sent): 1.23 ms

jerem@Jeremy: ~/timing_ws X + v
jerem@Jeremy:~$ cd ~/timing_ws
source /opt/ros/jazzy/setup.bash
source ~/timing_ws/install/setup.bash
python3 test_velocity_trigger.py --trials 5 --threshold 0.5
Trial 1: delay = 2.43 ms
Trial 2: delay = 3.37 ms
Trial 3: delay = 2.84 ms
Trial 4: delay = 2.47 ms
Trial 5: delay = 2.64 ms

--- Summary ---
Trials: 5
Range: 2.43ms - 3.37ms
Average: 2.75ms
jerem@Jeremy:~/timing_ws$ ^C
jerem@Jeremy:~/timing_ws$
```